

PROJECT 359 2024-2025

Εμψάλντο Λάμκια ΑΜ5059

Αντώνιος Καβαλιέρος ΑΜ5088



!ΠΡΟΣΟΧΗ!

Δείτε πρώτα το documents/installationInstructions/Manual για σωστή εγκατάσταση.
Επίσης η αρχική σας βάση χρειάζονται μικρο αλλαγές για να δουλέψει στο προτζεκτ
μας δείτε στο τέλος του αρχείου.

Τι Κάναμε:

Όλα από τον πινάκα με τις βαθμολογίες, τα μόνους και ένα μικρο παιχνιδάκι.

1. Τις κλάσεις και τις μεθόδους που χρησιμοποιήσατε:

Λόγω ότι χρησιμοποιήσαμε JavaScript (Node.js) αντί για Java δεν είχαμε κλάσεις. Γιαυτό θα θεωρήσουμε το κάθε .js αρχείο σαν μια κλάση.

AdminHome.js:

Το AdminHome.js είναι η το HomePage του Admin. Εκεί μπορεί να κάνει όλες τις λειτουργίες του. Συνδέεται με την AdminHome.html και την AdminHome.css.

Οι μέθοδοι της:

1. Φόρμα που φτιάχνει ο Admin.

viewLogIncident(): Κάνει την φόρμα ορατή.
hideLogIncident(): Κάνει την φόρμα μη ορατή.
Incident_Logging(): Στέλνει δεδομένα στη Βάση.
makeAjaxReq(): Φτιάχνει χάρτη.

2. Φόρμα για τα submitted που είτε κάνει real, είτε fake.

SubmittedCount(): Μετράει πόσα submitted υπάρχουν.
showSubmittedIncidents(): Στοιχεία της φόρμας.
SubmittedupdateDropdown(): Δείχνει στα πλάγια τα submitted που υπάρχουν.
view_update_message(): Κάνει την φόρμα ορατή.
hide_update_message(): Κάνει την φόρμα μη ορατή.
view_message(): Αν κάνει real, βγάζει επιλογές.
hide_message(): Αν δεν κάνει real.
Update_Submitted_Incident(): Στέλνει δεδομένα στη Βάση.
makeAjaxReqSub(): Φτιάχνει χάρτη.

3. Φόρμα για τα running που είτε κάνει update, είτε finished.

RunningCount(): Μετράει πόσα running υπάρχουν.
showRunningIncidents(): Στοιχεία της φόρμας.
RunningupdateDropdown: Δείχνει στα πλάγια τα running που υπάρχουν.
view_update_running_message(): Κάνει την φόρμα ορατή.
hide_update_running_message(): Κάνει την φόρμα μη ορατή.
validateAllFields(): Αν πατήσει finished και συμπληρώσει όλα τα πεδία βγάζει button.
End_Incident(): Κάνει την φόρμα για το finished.
Finished_Drivers(): Φέρνει τα στοιχεία των Driver που ενεργούν.
Finished_Simples(): Φέρνει τα στοιχεία των Simple που ενεργούν.
View_Request_Drivers(): Φέρνει τα στοιχεία των Driver που έχουν κάνει submitted.
View_Request_Simples(): Φέρνει τα στοιχεία των Simple που έχουν κάνει submitted.
Update_Running_Incident(): Στέλνει δεδομένα στη Βάση.
makeAjaxReqRun(): Φτιάχνει χάρτη.
Makeparticipants(what_form, Incident_Id): Φτιάχνει στη βάση θέσεις για participants.

4. Στατιστικά

pites(): Φτιάχνει πίτες.
columns(): Φτιάχνει στήλες.

RunnningAndFinished_Drivers_Simples(): Φέρνει στατιστικά για αυτούς.
Active_Drivers_Simples(): Επίσης. 5.Γενικές. 1,2,3,4.

AdminLogin.js:

Το AdminLogin.js είναι για να συνδέεται ο Admin στο AdminHome.js είναι μια απλή φόρμα.
Συνδέεται με την AdminLogIn.html και την AdminLogIn.css.

password_view(inputId, eyeId): Βλέπουμε τον κωδικό ή όχι.
submit_form(): Να δούμε αν είναι ο Admin.

Guest.js:

Το Guest.js είναι το HomePage των Guest δεν έχει LogIn γιατί είναι απλός Guest. Συνδέεται με την Guest.html και την Guest.css.

Οι μέθοδοι της:

1. Φόρμα που φτιάχνει ο Guest.

viewLogIncident(): Κάνει την φόρμα ορατή.
hideLogIncident(): Κάνει την φόρμα μη ορατή.
Incident_Logging(): Στέλνει δεδομένα στη Βάση.
toggleMap(): Φτιάχνει χάρτη.
toggleLogIncident(): Ανοιγοκλείνει την φόρμα.
askQuestion(question): Ερωτήσεις στο chat.

2. Τα υπόλοιπα στις γενικές.

Γενικές μέθοδοι που χρησιμοποιούνται συχνά:

1. Public message

viewPublicMessage(): Κάνει ορατή την φόρμα.
closePublicForm(): Κάνει μη ορατή την φόρμα.
PublicMessage(): Φέρνει όλα μηνύματα.
filterMessagesByIncident(): Ταξινομεί τα μηνύματα με βάση το Incident.
displayPublicMessages(messages): Εμφανίζει τα μηνύματα.
sendMessage(userType): Στέλνει μήνυμα.

2. User message

viewUserMessage(): Κάνει ορατή την φόρμα.
closeUserForm(): Κάνει μη ορατή την φόρμα.
UsersMessage(): Φέρνει όλα μηνύματα.
filterMessages(): Ταξινομεί τα μηνύματα με βάση τους users.
displayUserMessages(messages): Εμφανίζει τα μηνύματα.
sendUserMessage(userType): Στέλνει μήνυμα.

3. Volunteer message

viewVolunteerMessage(): Κάνει ορατή την φόρμα.
closeVolunteerForm(): Κάνει μη ορατή την φόρμα.
VolunteersMessage(): Φέρνει όλα μηνύματα.
updateVolunteersDropdown(incidentId): Ταξινομεί τα μηνύματα με βάση το Incident.

filterVolunteerMessages(): Ταξινομεί τα μηνύματα με βάση τους users.
displayVolunteerMessages(messages):Εμφανίζει τα μηνύματα.
checkVolunteerOption(): Τσεκάρει που στέλνει μήνυμα.
sendUserMessage(userType): Στέλνει μήνυμα.

4.History

viewHistory(): Κάνει ορατή την φόρμα.
displayHistory(history): Φέρνει τα αποτελέσματα.

5.Map for Active Incidents

showMapWithFires(): Φτιάχνει τον χάρτη.
createAccidentIcon(): Φτιάχνει icon με αυτοκίνητα.
createFireIcon(): Φτιάχνει icon με φωτιά.

code.js:

Το code.js είναι η 2 σειρά μας άσκηση λίγο αλλαγμένη. Χρησιμοποιείται για να κάνει register/login τους volunteers. Συνδέεται με την form.html που είναι η εμφάνιση της σελίδας και με την style.css
Οι μέθοδοι της:

checkcode(): Έλεγχος των κωδίκων από την 2η σειρά.
ShowPass(): Εμφανίζει το password.
ShowStrength():Εμφανίζει πόσο δυνατός είναι ο κωδικός (2η σειρά).
Checkrepeated():Έλεγχος για password repeats (2η σειρά).
Countnumbers():Μετράει πόσοι αριθμοί είναι στο passwords (2η σειρά).
Allowedwords():Έλεγχος για επιτρεπτές λέξεις στο password (2η σειρά).
ContainsSpecialChars():Έλεγχος αν υπάρχουν special characters (2η σειρά).
Checkstrong():Έλεγχος πόσο δυνατός είναι ο κωδικός (2η σειρά).
Passafety():Έλεγχος από την 1 σειρά.
SetPosition(),handler(),makeAjaxReq():Μέθοδος για το API των map(2η σειρά).
CheckEmail/Username/Phone():Έλεγχος για email,phone username από την βάση runtime.
Onlist():Έλεγχος αν τα παραπάνω ισχύουν και κάνει submit την φόρμα και την στέλνει στην βάση.
Home():Σε πάει στο home.html
login():Εμφανίζει το login page run time σε single page.
LogIn():Σε συνδέει στο Volunteers.
RegisterAgain():Εμφανίζει την φόρμα για εγγραφή.

volunteers.js:

Το volunteers.js είναι το Home Page των Volunteers. Χρησιμοποιείται για να τις λειτουργίες των volunteers όπως να κάνουν αίτηση για incidents ή να επικοινωνήσουν. Συνδέεται με την form.html που είναι η εμφάνιση της σελίδας και με την style.css
Οι μέθοδοι της:

showUpdateForm():Εμφανίζει την φόρμα με τα στοιχεία του.
HideForm():Κρύβει την φόρμα με τα στοιχεία του.
Update():κάνει update τα στοιχεία του.
LoggedIn():Έλεγχος αν είναι συνδεδεμένος και ενημερώνει το carousel.

TakePlaceNow():Στέλνει αίτηση για συμμετοχή στο incidents.
ShowRunningIncidents():Εμφανίζει όλα τα incidents που συμμετέχει.
ShowFinishedIncidents(): Εμφανίζει όλα τα incidents που είχε συμμετέχει.
Τα υπόλοιπα υπάρχουν ήδη πάνω.

UserForm.js:

Το UserForm.js είναι η 2 σειρά μας άσκηση λίγο αλλαγμένη όπως το code.js. Χρησιμοποιείται για να κάνει register/login τους users. Συνδέεται με την UserForm.html που είναι η εμφάνιση της σελίδας και με την UserForm.css.

Οι μέθοδοι της:

Είναι ίδια με την code.js άπλα αλλάζει στο ποιο πινάκω να κοιτάει στην Βάση.

User.js:

Το User.js είναι το HomePage των Users. Είναι σαν τους Volunteers άλλα χωρίς τα incidents. Συνδέεται με User.html και User.css.

Οι μέθοδοι της:

GetName():Παίρνει το όνομα του χρήστη και ελέγχει αν όντως είναι συνδεδεμένος.

CalculateDistance():Υπολογίζει την απόσταση με όλα τα incidents αν υπάρχει κίνδυνος nearby.

Τα υπόλοιπα υπάρχουν από πάνω.

2. Τη γενικότερη αρχιτεκτονική του συστήματος. Το σχέδιο σας είναι καλό να απαντάει:

1.Πόσα, ποια και με τι αρμοδιότητα servlets χρησιμοποιήσατε (η ότι άλλη τεχνολογία χρησιμοποιήσατε):

Χρησιμοποιήσαμε Node.js με express αντί για servlets. Όλο το backEnd requests γίνονται στο db.js που ενώνεται με την βάση μας. Πόσα και ποια απαντιούνται παρακάτω.

2.Πόσα, ποια και με τι αρμοδιότητα JavaScript Libraries/html χρησιμοποιήσατε:

axios:Το χρησιμοποιήσαμε στην αρχή ήταν για request στου server.

Cookie-parser:Το χρησιμοποιήσαμε για τα cookies.

Cors:Λόγω ασφάλειας λίγο άχρηστο στο πρότζεκτ μας άλλα το κρατήσαμε.

Dbgate-query-splitter:βιβλιοθήκη που παρέχει τη δυνατότητα να διαχωρίζονται SQL queries

μέσα σε ένα string ή αρχείο. Δεν το χρησιμοποιήσαμε στο τέλος του πρότζεκτ.

Dotenv:Είναι για να βάζουμε τα στοιχεία μας ξεχωριστά χωρίς να χρειάζεται να αλλάξουμε όλων των κωδικά.

Express:Είναι απαραίτητη framework για την χρήση των request HTTP RESTAPI κτλπ.

Mysql && mysql2 :Χρήσιμη για την επικοινωνία με την βάση μας.

Nodemon:Είναι μια απλή βιβλιοθήκη που τρέχει τον server μας συνεχεία χωρίς να χρειάζεται να τον σταματάμε σε κάθε αλλαγή άλλα αυτό το κάνει μόνο του.

Openai:Το chatgpt API που εν τέλη δεν το χρησιμοποιήσαμε γιατί δεν υπήρχε δωρεάν. Αντί για αυτό όμως χρησιμοποιήσαμε το @google/generative-ai που είναι το Gemini AI της google.

Path:Βιβλιοθήκη για τα paths μας.

Bootstrap: Χρησιμοποιήσαμε Bootstrap σε FrontEnd για να είναι reactive άλλα και εύκολη στην υλοποίηση μας.

3.Πόσα, ποια και με τι αρμοδιότητα css χρησιμοποιήσατε:

font-family:Ποια γραμματοσειρά να βάλει.

background:Στυλ Background.

margin:Κενό γύρω από το στοιχείο.

color:Χρώμα του κειμένου.

background-color:Χρώμα background.

padding:Κενό μέσα στο στοιχείο.

box-shadow:Βάζει σκιά σε κουτί που έχει μέσα το στοιχείο.

display:Τρόπος εμφάνισης.

align-items:Βάζει τα στοιχεία όπως του τα ζητήσουμε.

font-size:Μέγεθος γραμματοσειράς.

margin-right:Κενό γύρω στο στοιχείο από δεξιά.

margin-top:Κενό γύρω στο στοιχείο από πάνω.

gap:Κενό σε align στοιχεία.

padding-left:Προσθέτει κενό μέσα στο στοιχείο από αριστερά.

font-weight:Πάχος γραμματοσειράς.

border-radius:Πως να φαίνονται οι γωνίες του border.

transition:Πως θα φαίνονται οι αλλαγές για τα animation.

white-space:Τι θα κάνει με το κενό space μέσα σε ένα grid.

transform:Μετακίνηση ενός στοιχείου

max-width:Μέγιστο πλάτος.

text-align:Align το κείμενο σε ένα grid.

margin-bottom:Προσθέτει κενό από κάτω.

resize:Αλλαγή μεγεθύνσου ενός στοιχείου.

height:Ύψος ενός στοιχείου.

width:Πλάτος ενός στοιχείου.

border:Ορίζει border.

outline:Το περίγραμμα ενός στοιχείου.

cursor:Τύπος του δείκτη.

overflow:Ελέγχει τη συμπεριφορά του περιεχομένου όταν ξεφεύγει από το στοιχείο
overflow-y:Ελέγχει την συμπεριφορά του περιεχομένου όταν ξεφεύγει από το στοιχείο προς τον y άξονα
word-wrap:Ορίζει πώς να χειρίζονται οι μεγάλες λέξεις όταν ξεφεύγουν από το πλαίσιο.
flex:Οι ιδιότητες flex ενός στοιχείου (flex-grow, flex-shrink, flex-basis).
opacity:Ορίζει το opacity ενός στοιχείου.
justify-content:Κάνει align τα στοιχεία x άξονα.
list-style-type:Ορίζει το στυλ των στοιχείων σε μια λίστα.
border-bottom:Προσθέτει ένα κάτω border σε ένα στοιχείο.
letter-spacing:Το κενό μεταξύ των γραμμάτων του κειμένου.
text-shadow:Προσθέτει σκιά στο κείμενο.
backdrop-filter:Εφαρμόζει γραφικά εφέ στην περιοχή πίσω από ένα στοιχείο.
border-color:Ορίζει το χρώμα του border ενός στοιχείου.
padding-left:Κενό μέσα στο στοιχείο από αριστερά.
animation:Ορίζει keyframes για animation
gradientBackground:Βάζει κινούμενο background.
background-position:Βάζει την θέση του Background.
line-height:Ελέγχει το κενό μεταξύ των γραμμών του κειμένου.
box-sizing:Ορίζει πώς υπολογίζεται το συνολικό πλάτος-ύψος ενός στοιχείου
text-decoration:Βάζει υπογράμμιση, πάνω από γραμμή και άλλα τέτοια.
grid-template-columns:Ορίζει τις στήλες σε ένα grid layout.
flex-direction:Ορίζει τη κατεύθυνση των αντικειμένων σε ένα flex container

Η πλευρά του client με javascript && Τα Ajax τμήματα της εφαρμογής σας && REST Request && Τα BackEnd Request:

!Χρησιμοποιήσαμε fetch αντί για XMLHttpRequest!

1.Ο client ζητάει να μάθει πόσα incidents είναι submitted.

Ajax

```
const response = await fetch('http://localhost:3000/getSubmittedCount', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getSubmittedCount', (req, res)
```

2.Ο client ζητάει τα στοιχεία από τα incidents που είναι submitted.

Ajax

```
const res = await fetch('http://localhost:3000/getSubmittedIncidents', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getSubmittedIncidents', (req, res)
```

3.Ο client ζητάει να μάθει πόσα incidents είναι running.

Ajax

```
const response = await fetch('http://localhost:3000/getRunningCount', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getRunningCount', (req, res)
```

4.Ο client ζητάει τα στοιχεία από τα incidents που είναι running.

Ajax

```
const res = await fetch('http://localhost:3000/getRunningIncidents', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getRunningIncidents', (req, res)
```

5.Ο client ζητάει να μάθει πόσοι Drivers υπάρχουν σε ένα incident που είναι running.

Ajax με query part IncidentID

```
const IncidentId = incidents.incident_id;  
const response = await fetch(`http://localhost:3000/getCountDriversRunning?IncidentID=${  
{IncidentId}}`, {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getCountDriversRunning', (req, res) => {  
  const { IncidentID } = req.query;
```

6.Ο client ζητάει να μάθει πόσοι Simples υπάρχουν σε ένα incident που είναι running.

Ajax με query part IncidentID

```
const IncidentId = incidents.incident_id;  
const response2 = await fetch(`http://localhost:3000/getCountSimplesRunning?IncidentID=${  
{IncidentId}}`, {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getCountSimplesRunning', (req, res) => {  
  const { IncidentID } = req.query;
```

7.Ο client Βάζει στην βάση ένα incident.

Ajax

```
const res = await fetch("http://localhost:3000/AddAdminIncident", {  
  method: "POST",
```



```

    credentials: "include",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(formData),
  });
REST requests
app.post("/AddAdminIncident", (req, res)
  const {
    incident_type,
    description,
    user_phone,
    user_type,
    address,
    lat,
    lon,
    municipality,
    prefecture,
    start_datetime,
    end_datetime,
    danger,
    status,
    finalResult,
    vehicles,
    firemen
  } = req.body;

```

8.O client Θέλει τα id των incident.

Ajax με query part Description και Startday

```

const response = await fetch(`http://localhost:3000/getAdminIncidentID?Description=${
  document.getElementById("IncidentDescription").value}&Startday=${now}`, {
  method: 'GET',
  credentials: 'include'
});

```

REST requests

```

app.get('/getAdminIncidentID', (req, res) => {
  const { Description, Startday } = req.query;

```

9.O client ζητάει τα στοιχεία των Drivers που είναι ενεργοί σε ένα συμβάν.

Ajax με query part IncidentID

```

const response2 = await fetch(`http://localhost:3000/getDriversRunning?IncidentID=${IncidentId}`,
{
  method: 'GET',
  credentials: 'include'
});

```

REST requests

```

app.get('/getDriversRunning', (req, res) => {
  const { IncidentID } = req.query;

```

10.Ο client ζητάει τα στοιχεία των Simple που είναι ενεργοί σε ένα συμβάν.

Ajax με query part IncidentID

```
const response2 = await fetch(`http://localhost:3000/getSimplesRunning?IncidentID=${IncidentId}`,  
{  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getSimplesRunning', (req, res) => {  
  const { IncidentID } = req.query;
```

11.Ο client ζητάει τα στοιχεία των volunteer με ένα συγκεκριμένο όνομα.

Ajax με query part VolunteerUsername

```
const VolunteerUsername = driver.volunteer_username;  
const res = await fetch(`http://localhost:3000/getVolunteerDetails?VolunteerUsername=${  
{VolunteerUsername}`, {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getVolunteerDetails', (req, res) => {  
  const { VolunteerUsername } = req.query;
```

11.Ο client ενημερώνει ένα συγκεκριμένο συμβάν.

Ajax με query part IncidentID

```
const IncidentId = document.getElementById("ID").textContent.trim();  
const response = await fetch(`http://localhost:3000/EndIncident?IncidentID=${IncidentId}`, {  
  method: 'PUT',  
  headers: {  
    'Content-Type': 'application/json'  
  },  
  credentials: 'include',  
  body: JSON.stringify({  
    end_datetime: now,  
    status: "finished",  
    finalResult: finalResult,  
    incident_id: IncidentId  
  })  
});
```

REST requests

```
app.put("/EndIncident", (req, res) => {  
  const { end_datetime, status, finalResult, incident_id } = req.body;
```

12.Ο client ενημερώνει τους participants για ένα συγκεκριμένο συμβάν.

Ajax με query part IncidentID

```
const response2 = await fetch(`http://localhost:3000/EndParticipants?IncidentID=${IncidentId}`, {  
  method: 'PUT',  
  headers: {
```

```

    'Content-Type': 'application/json'
  },
  credentials: 'include',
  body: JSON.stringify({
    status: "finished",
    success: "YES",
    comment: commentsData[i].comment,
    volunteer_username: commentsData[i].username,
    incident_id: IncidentId,
    volunteer_type: commentsData[i].type,
  })
});
REST requests
app.put("/EndParticipants", (req, res) => {
  const { status, success, comment, volunteer_username, incident_id, volunteer_type } = req.body;

```

13.Ο client ζητάει πόσοι Drivers είναι σε αναμονή σε ένα συμβάν.

```

Ajax με query part IncidentID
const IncidentId = document.getElementById("ID").textContent.trim();
const response = await fetch(`http://localhost:3000/getCountDriversRequest?IncidentID=${IncidentId}`, {
  method: 'GET',
  credentials: 'include'
});
REST requests
app.get('/getCountDriversRequest', (req, res) => {
  const { IncidentID } = req.query;

```

14.Ο client ζητάει πόσοι Simples είναι σε αναμονή σε ένα συμβάν.

```

Ajax με query part IncidentID
const IncidentId = document.getElementById("ID").textContent.trim();
const response = await fetch(`http://localhost:3000/getCountSimplesRequest?IncidentID=${IncidentId}`, {
  method: 'GET',
  credentials: 'include'
});
REST requests
app.get('/getCountSimplesRequest', (req, res) => {
  const { IncidentID } = req.query;

```

15.Ο client ζητάει τα στοιχεία των Drivers που είναι σε αναμονή σε ένα συμβάν

```

Ajax με query part IncidentID
const IncidentId = document.getElementById("ID").textContent.trim();
const response = await fetch(`http://localhost:3000/getDriversRequest?IncidentID=${IncidentId}`, {
  method: 'GET',
  credentials: 'include'
});
REST requests
app.get('/getDriversRequest', (req, res) => {

```

```
const { IncidentID } = req.query;
```

16.Ο client ζητάει τα στοιχεία των Simple που είναι σε αναμονή σε ένα συμβάν

Ajax με query part IncidentID

```
const IncidentId = document.getElementById("ID").textContent.trim();
const response = await fetch(`http://localhost:3000/getSimpleRequest?IncidentID=${IncidentId}`, {
  method: 'GET',
  credentials: 'include'
});
```

REST requests

```
app.get('/getSimpleRequest', (req, res) => {
  const { IncidentID } = req.query;
```

17.Ο client ενημερώνει τους participants για ένα συγκεκριμένο συμβάν ότι αποδέχεται τον εθελοντή.

Ajax

```
const response1 = await fetch(`http://localhost:3000/acceptRequest`, {
  method: 'PUT',
  headers: {
    'Content-Type': 'application/json'
  },
  credentials: 'include',
  body: JSON.stringify({
    status: "accepted",
    participant_id: driver.participant_id
  })
});
```

REST requests

```
app.put("/acceptRequest", (req, res) => {
  const { status, participant_id } = req.body;
```

18.Ο client ενημερώνει τους participants για ένα συγκεκριμένο συμβάν ότι δεν αποδέχεται τον εθελοντή.

Ajax

```
const response2 = await fetch(`http://localhost:3000/declineRequest`, {
  method: 'PUT',
  headers: {
    'Content-Type': 'application/json'
  },
  credentials: 'include',
  body: JSON.stringify({
    volunteer_username: "null",
    participant_id: driver.participant_id
  })
});
```

REST requests

```
app.put("/declineRequest", (req, res) => {
  const { volunteer_username, participant_id } = req.body;
```

19.Ο client ενημερώνει ένα συγκεκριμένο συμβάν ότι είναι real.

Ajax

```
const res2 = await fetch("http://localhost:3000/AdminUpdateSubmittedIncident", {  
  method: "PUT",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify(formData),  
});
```

REST requests

```
app.put("/AdminUpdateSubmittedIncident", (req, res) => {  
  const {  
    incident_id,  
    incident_type,  
    description,  
    address,  
    lat,  
    lon,  
    municipality,  
    prefecture,  
    start_datetime,  
    danger,  
    status,  
    vehicles,  
    firemen  
  } = req.body;
```

20.O client ενημερώνει ένα συγκεκριμένο συμβάν ότι είναι fake.

Ajax

```
const res2 = await fetch("http://localhost:3000/AdminFakeIncident", {  
  method: "PUT",  
  headers: { "Content-Type": "application/json" },  
  body: JSON.stringify(formData),  
});
```

REST requests

```
app.put("/AdminFakeIncident", (req, res) => {  
  const {incident_id, status} = req.body;
```

21.O client ζητάει το vehicles και το firemen

Ajax με query part IncidentID

```
const response = await fetch(`http://localhost:3000/getVehiclesAndFiremen?IncidentID=${  
  IncidentID}`, {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getVehiclesAndFiremen', (req, res) => {  
  const { IncidentID } = req.query;
```

22.O client ενημερώνει ένα συγκεκριμένο συμβάν με νέα στοιχεία

Ajax

```
const res2 = await fetch("http://localhost:3000/AdminUpdateRunningIncident", {
```

```
method: "PUT",
headers: { "Content-Type": "application/json" },
body: JSON.stringify(formData),
});
```

REST requests

```
app.put("/AdminUpdateRunningIncident", (req, res) => {
  const { incident_id, description, danger, vehicles, firemen } = req.body;
```

23.O client φτιάχνει θέσεις για Drivers.

Ajax

```
const res = await fetch("http://localhost:3000/CreateDriverParticipant", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify(formData),
});
```

REST requests

```
app.post("/CreateDriverParticipant", (req, res) => {
  const { incident_id } = req.body;
```

23.O client φτιάχνει θέσεις για Simples.

Ajax

```
const res = await fetch("http://localhost:3000/CreateSimpleParticipant", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify(formData),
});
```

REST requests

```
app.post("/CreateSimpleParticipant", (req, res) => {
  const { incident_id } = req.body;
```

24.O client ζητάει τον αριθμό των ενεργών πυρκαγιών.

Ajax

```
const res = await fetch('http://localhost:3000/getCountRunningFireIncidents', {
  method: 'GET',
  credentials: 'include'
});
```

REST requests

```
app.get('/getCountRunningFireIncidents', (req, res)
```

24.O client ζητάει τον αριθμό των ενεργών ατυχημάτων.

Ajax

```
const res2 = await fetch('http://localhost:3000/getCountRunningAccidentIncidents', {
  method: 'GET',
  credentials: 'include'
});
```

REST requests

```
app.get('/getCountRunningAccidentIncidents', (req, res)
```

24.Ο client ζητάει τον αριθμό των τερματισμένων πυρκαγιών.

Ajax

```
const res3 = await fetch('http://localhost:3000/getCountFinishedFireIncidents', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getCountFinishedFireIncidents', (req, res)
```

25.Ο client ζητάει τον αριθμό των τερματισμένων ατυχημάτων.

Ajax

```
const res4 = await fetch('http://localhost:3000/getCountFinishedAccidentIncidents', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getCountFinishedAccidentIncidents', (req, res)
```

26.Ο client ζητάει τον αριθμό των χρηστών.

Ajax

```
const res = await fetch('http://localhost:3000/getCountUsers', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getCountUsers', (req, res)
```

26.Ο client ζητάει τον αριθμό των εθελοντών.

Ajax

```
const res2 = await fetch('http://localhost:3000/getCountVolunteers', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getCountVolunteers', (req, res)
```

27.Ο client ζητάει τον αριθμό των εθελοντών που είναι ενεργοί και έχουν τελειώσει και είναι Drivers.

Ajax

```
const res = await fetch('http://localhost:3000/getCountRFDDrivers', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getCountRFDDrivers', (req, res)
```

28.Ο client ζητάει τον αριθμό των εθελοντών που είναι ενεργοί και έχουν τελειώσει και είναι Simple.

Ajax

```
const res2 = await fetch('http://localhost:3000/getCountRFSimples', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getCountRFSimples', (req, res)
```

29.O client ζητάει τον αριθμό των εθελοντών που είναι ενεργοί και είναι Drivers.

Ajax

```
const res = await fetch('http://localhost:3000/getCountADrivers', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getCountADrivers', (req, res)
```

30.O client ζητάει τον αριθμό των εθελοντών που είναι ενεργοί και είναι Simples.

Ajax

```
const res2 = await fetch('http://localhost:3000/getCountADSimples', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getCountADSimples', (req, res)
```

31.O client ζητάει τα Public messages.

Ajax

```
const response = await fetch('http://localhost:3000/getPublicMessage', {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/getPublicMessage', (req, res)
```

32.O client ζητάει να δει αν ο sender είναι volunteer.

Ajax με query part username

```
const res = await fetch(`http://localhost:3000/IsVolunteer?username=${message.sender}`, {  
  method: 'GET',  
  credentials: 'include'  
});
```

REST requests

```
app.get('/IsVolunteer', (req, res) => {  
  const { username } = req.query;
```

33.O client ζητάει να δει αν ο sender είναι user.

Ajax με query part username

```
const res2 = await fetch(`http://localhost:3000/IsUser?username=${message.sender}`, {  
  method: 'GET',
```



```

        credentials: 'include'
    });
REST requests
app.get('/IsUser', (req, res) => {
    const { username } = req.query;

```

34.O client βάζει νέο μήνυμα.

Ajax

```

const res5 = await fetch("http://localhost:3000/AddMessage", {
    method: "POST",
    credentials: "include",
    headers: {
        "Content-Type": "application/json",
    },
    body: JSON.stringify({
        incident_id: selectedIncident,
        message: messageContent,
        sender: "admin",
        recipient: "public",
        date_time: now
    }),
});

```

REST requests

```

app.post("/AddMessage", (req, res) => {
    const { incident_id, message, sender, recipient, date_time } = req.body;

```

35.O client ζητάει τα User messages.

Ajax

```

const response = await fetch('http://localhost:3000/getUserMessage', {
    method: 'GET',
    credentials: 'include'
});

```

REST requests

```

app.get('/getUserMessage', (req, res)

```

36.O client ζητάει τα στοιχεία των Users .

Ajax

```

const res = await fetch('http://localhost:3000/getUsers', {
    method: 'GET',
    credentials: 'include'
});

```

REST requests

```

app.get('/getUsers', (req, res)

```

37.O client ζητάει τα Volunteer messages.

Ajax

```

const response = await fetch('http://localhost:3000/getVolunteersMessage', {
    method: 'GET',

```

```
    credentials: 'include'
  });
REST requests
app.get('/getVolunteersMessage', (req, res)
```

38.Ο client ζητάει τα στοιχεία των Volunteers .

Ajax

```
const res = await fetch('http://localhost:3000/getVolunteers', {
  method: 'GET',
  credentials: 'include'
});
```

REST requests

```
app.get('/getVolunteers', (req, res)
```

38.Ο client ζητάει τους Volunteers για ένα συγκεκριμένο συμβάν.

Ajax

```
const res2 = await fetch(`http://localhost:3000/getVolunteersByIncident?incidentId=${incidentId}`, {
  method: 'GET',
  credentials: 'include'
});
```

REST requests

```
app.get('/getVolunteersByIncident', (req, res) => {
  const { incidentId } = req.query;
```

39.Ο client ζητάει τους Volunteers για ένα συγκεκριμένο συμβάν.

Ajax με query parts

```
const res = await fetch(`http://localhost:3000/getHistory?${queryParams}`, {
  method: 'GET',
  credentials: 'include',
});
```

REST requests

```
app.get('/getHistory', async (req, res) => {
  const { prefecture, type, start_datetime, end_datetime, vehicles, firemen } = req.query;
```

40.Ο client ζητάει απαντήσεις από το chat.

Ajax

```
const res = await fetch('http://localhost:3000/getChat', {
  method: 'POST',
  credentials: 'include',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({ prompt })
});
```

REST requests

```
app.post("/getChat", async (req, res) => {
  const { prompt } = req.body;
```

41.Ο client ζητάει Lat και το Lon.

Ajax

```
const res = await fetch('http://localhost:3000/GetLatLonUser', {
  method: 'GET',
  credentials: 'include'
});
```

REST requests

```
app.get("/GetLatLonUser", (req, res) => {
  const user_id = req.cookies.user_id;
```

42.Ο client ζητάει Lat και το Lon και όλες τις πληροφορίες του.

Ajax

```
const res2 = await fetch('http://localhost:3000/GetLatLonIncidents', {
  method: 'GET',
  credentials: 'include'
});
```

REST requests

```
app.get("/GetLatLonIncidents", (req, res)
```

43.Ο client υπολογίζει την απόσταση του user με όλα τα incidents.

Ajax

```
const url = `https://trueway-matrix.p.rapidapi.com/CalculateDrivingMatrix?origins=${
encodeURIComponent(userlat + ',' + userlon)}&destinations=${encodeURIComponent(data2[i].lat +
',' + data2[i].lon)}`;
const options = {
  method: 'GET',
  headers: {
    'x-rapidapi-key': '/...../',
    'x-rapidapi-host': 'trueway-matrix.p.rapidapi.com'
  }
};
```

REST requests

API

44.Ο client ελέγχει αν υπάρχει κάποιος άλλος στην βάση με το ίδιο Username.

Ajax με query part Username

```
const res = await fetch("http://localhost:3000/CheckUserNameUser?Username=" + username, {
  method: "GET",
  credentials: "include",
});
```

REST requests

```
app.get("/CheckUserNameUser", (req, res) => {
  const Username = req.query.Username;
```

45.Ο client ελέγχει αν υπάρχει κάποιος άλλος στην βάση με το ίδιο Email.

Ajax με query part Email

```
const res = await fetch("http://localhost:3000/CheckEmailUser?Email=" + email, {
  method: "GET",
```

```
    credentials: "include",
  });
REST requests
app.get("/CheckEmailUser", (req, res) => {
  const Email = req.query.Email;
```

46.Ο client ελέγχει αν υπάρχει κάποιος άλλος στην βάση με το ίδιο PhoneNumber.

Ajax με query part PhoneNumber

```
const res = await fetch("http://localhost:3000/CheckPhoneUser?PhoneNumber=" + telephone, {
  method: "GET",
  credentials: "include",
});
```

REST requests

```
app.get("/CheckPhoneNumbeUser", (req, res) => {
  const PhoneNumbe = req.query.PhoneNumbe;
```

47.Ο client προσθέτει user.

Ajax

```
const res = await fetch("http://localhost:3000/AddUser", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify(dataObject),
  credentials: "include",
});
```

REST requests

```
app.post("/AddUser", (req, res) => {
  const { username, email, password, firstname, lastname, birthdate, gender, afm, country, address,
  municipality, prefecture, job, telephone, lat, lon } = req.body;
```

48.Ο client ελέγχει αν είναι σωστά τα στοιχεία στον user.

Ajax

```
const res = await fetch("http://localhost:3000/loginUsers", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: JSON.stringify(dataObject),
});
```

REST requests

```
app.post("/loginUsers", (req, res) => {
  const { username, password } = req.body;
```

49.Ο client προσθέτει Volunteer.

Ajax

```
const res = await fetch("http://localhost:3000/AddVolunteer", {
  method: "POST",
  headers: {
```

```

        "Content-Type": "application/json",
    },
    body: JSON.stringify(dataObject),
    credentials: "include",
  });
  REST requests
  app.post("/AddVolunteer", (req, res) => {
    const { username, email, password, firstname, lastname, birthdate, gender, afm, country, address,
    municipality, prefecture, job, telephone, lat, lon, volunteer_type, height, weight } = req.body;

```

50.Ο client ελέγχει αν είναι σωστά τα στοιχεία στον volunteer.

Ajax

```

    const res = await fetch("http://localhost:3000/loginVolunteers", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify(dataObject),
    });

```

REST requests

```

app.post("/loginVolunteers", (req, res) => {
  const { username, password } = req.body;

```

51.Ο client ζητάει τα στοιχεία του volunteer.

Ajax

```

    const res = await fetch('http://localhost:3000/VolunteerData', {
      method: 'GET',
      headers: {
        'Content-Type': 'application/json',
      },
    });

```

REST requests

```

app.get("/VolunteerData", (req, res) => {
  const volunteer_id = req.cookies.volunteer_id;

```

52.Ο client ενημερώνει τον volunteer.

Ajax

```

    const res = await fetch('http://localhost:3000/UpdateVolunteer', {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify(dataObject),
    });

```

REST requests

```

app.put("/UpdateVolunteer", (req, res) => {
  const volunteer_id = req.cookies.volunteer_id;

```

```
const { username, email, telephone, address, password, firstname, lastname, birthdate, afm, country, municipality, prefecture, job, gender, height, weight } = req.body;
```

53.Ο client ζητάει όλα τα incidents.

Ajax

```
const res = await fetch('http://localhost:3000/GetIncidents', {
  method: 'GET',
  headers: {
    'Content-Type': 'application/json',
  },
});
```

REST requests

```
app.get("/GetIncidents", (req, res) => {
  const volunteer_id = req.cookies.volunteer_id;
```

54.Ο client ενημερώνει τους participants.

Ajax

```
const res = await fetch('http://localhost:3000/Addparticipant', {
  method: 'PUT',
  headers: {
    'Content-Type': 'application/json',
  },
  body: JSON.stringify({ incident_id: incident_id }),
});
```

REST requests

```
app.put("/Addparticipant", (req, res) => {
  const volunteer_id = req.cookies.volunteer_id;
  const { incident_id } = req.body;
```

Αν χρειάστηκε να κάνετε αλλαγές/προσθήκες στο σχήμα της βάσης:

Χρειάστηκε γιατί στην αρχική σας βάση τα participants ήταν διαφορετική λογική από την δίκη μας. Σε μας εμείς φτιάχνουμε participants με βάση ποσά firemen και vehicles χρειάζονται αφού έχουν γίνει running από τον Admin (ή έχει γίνει κάποιο update του incident από τον Admin). Αυτό όμως δεν γίνεται στην δίκη σας αρχική βάση με αποτέλεσμα να μην λειτουργεί σωστά όπως το κάναμε. Η δίκη μας λύση σε αυτό είναι να διαγράψουν όλα τα αρχικά participants που έχετε.

Delete FROM hy359_2024.participants WHERE participant_id = 3

Delete FROM hy359_2024.participants WHERE participant_id = 1

Delete FROM hy359_2024.participants WHERE participant_id = 2

!ΠΡΟΣΟΧΗ!

Sources: Οι εικόνες είναι AI Generated